01211373 Machine Learning and Programming for Industry

Lecture 10 Supplement 2

Recurrent networks on a mobile robot

Heading, terrain and wheel slip from IMU and wheel-encoder sequences

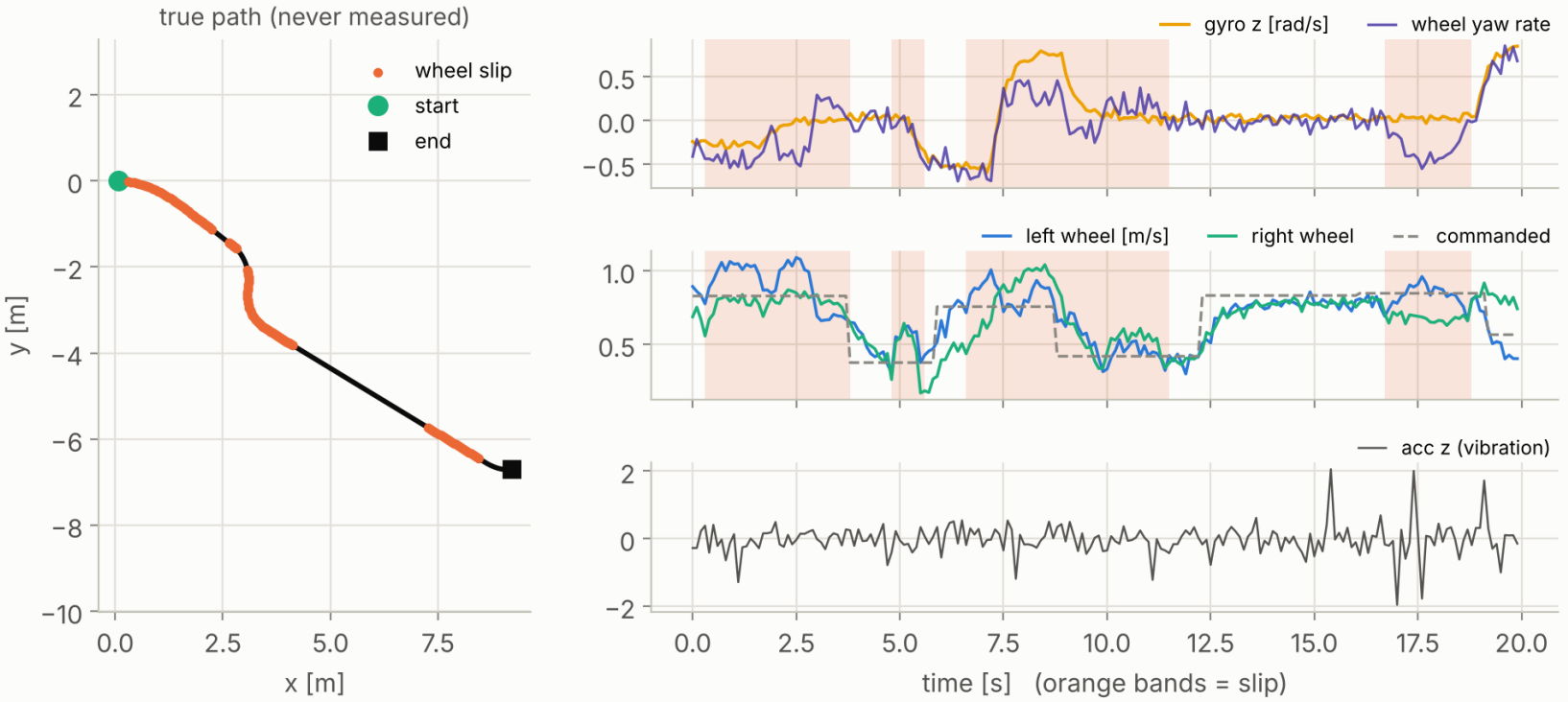
- Dead reckoning is already a recurrent network
- Sequence shapes: many-to-one and many-to-many
- Vanilla RNN vs LSTM vs GRU, and the measured gradient reach
- What a trained LSTM keeps in its cell state
- Ragged runs: pad, pack, or bucket
- Causal vs bidirectional, and the physics baselines to beat

Notebook: mlpi26_Lect10_nbsup2.ipynb
Companion to the CNC-cell lecture on ch. 15 (Raschka, Liu & Mirjalili)

THE RUNNING EXAMPLE

One rover, three sequence questions

- Seven channels at 10 Hz: commanded speed and turn, two wheel encoders, a yaw gyro and two accelerometers. Runs last 8–20 s on concrete, grass or gravel.
- Every sensor lies in its own way: encoders over-read when a wheel slips (20.5 % of steps), the gyro has a bias that changes every run, and acc_z mixes vibration with noise.



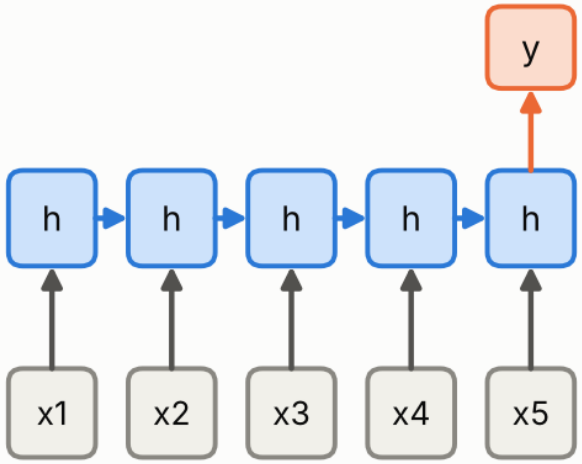
Questions: what floor is this (one label per run)? Is a wheel slipping now? Where is the robot pointing now?

Pick the shape before you pick the model

- The question fixes the shape. One answer per run is many-to-one: read the output at the last step. One answer per step is many-to-many: put a head on every hidden state.
- Heading and slip are both many-to-many here, but heading needs the whole past while slip needs only the present. Same shape, very different memory demand.

many-to-one

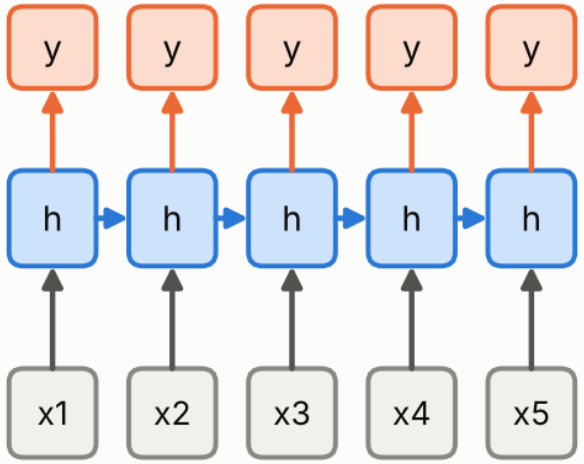
What terrain is this?



1 label per run

many-to-many, synchronised

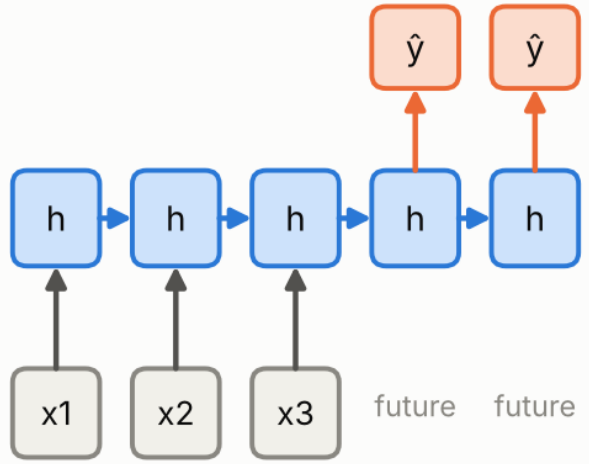
Is a wheel slipping now? Heading now?



1 output per step

sequence-to-sequence

Where will it be in 8 s?



Supplement B

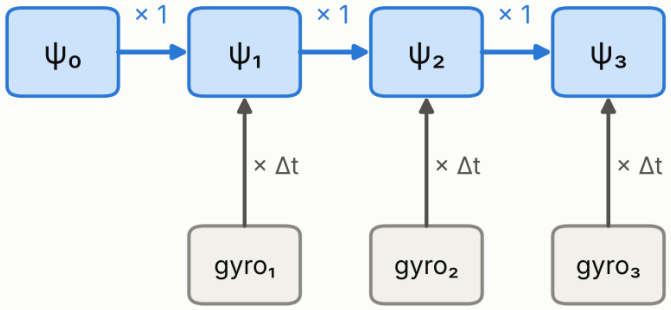
Shape = where the outputs are. Memory = how far back each output must look. They are separate decisions.

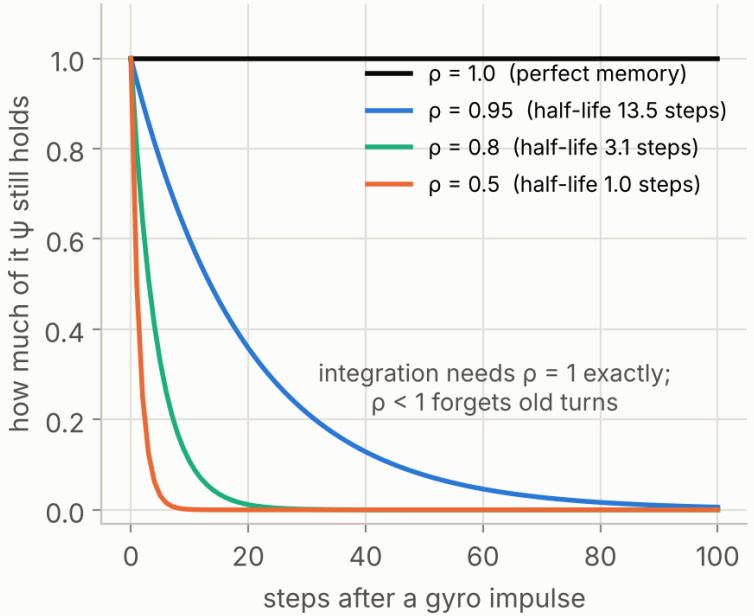
THE RECURRENCE

Dead reckoning is already a recurrent network

- Drop the tanh, set one hidden unit, W_hh = 1 and W_xh = Δt, and the RNN update becomes the heading integrator $\psi_t = \psi_{t-1} + \Delta t \cdot \text{gyro}_t$.
- A recurrent weight below 1 makes memory fade as ρ^k. Integration needs exactly 1, which a tanh network has to learn to hold without exploding.

$\psi_t = 1 \cdot \psi_{t-1} + \Delta t \cdot \text{gyro}_t$ — an RNN with W_hh = 1 and no tanh






```
h = 0.0
for g in gyro:
    h = 1.0*h + DT*g

# == np.cumsum(gyro)*DT
# max |diff| 9.5×10⁻7

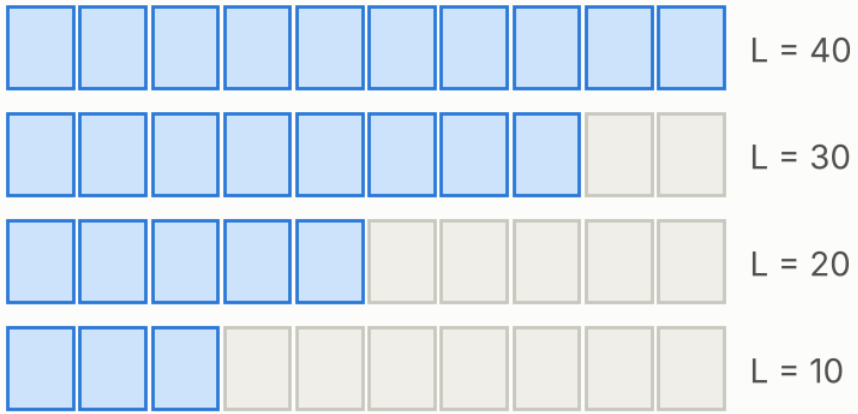
# tanh RNN by hand vs nn.RNN:
# max |diff| 1.5×10⁻7
```

The hidden state is a state estimate. A recurrent network learns the update f in $x_t = f(x_{t-1}, u_t)$.

Ragged runs: pad, pack, or bucket

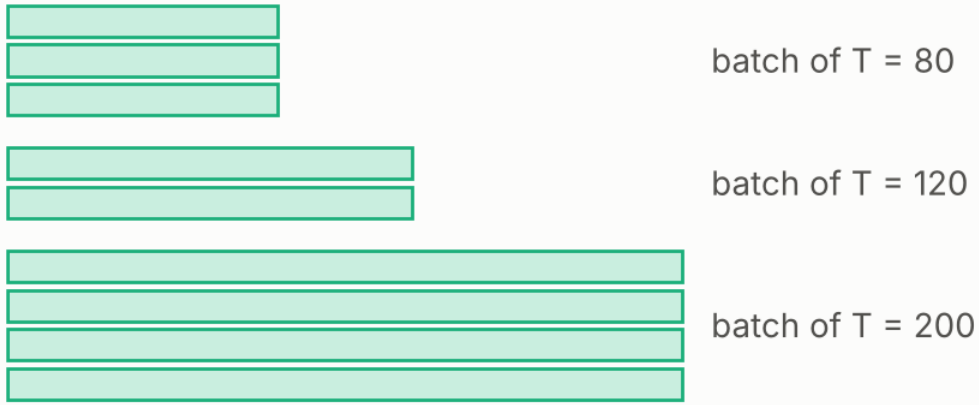
- A forward RNN's output at step L−1 is unaffected by padding that comes after it. The final state h_n and every bidirectional output are not.

pad to the longest run



38% of this batch is padding; the backward direction of a BiLSTM reads it first unless you pack

bucket runs of equal length



no padding at all → no packing, no masks, ~9× faster on CPU

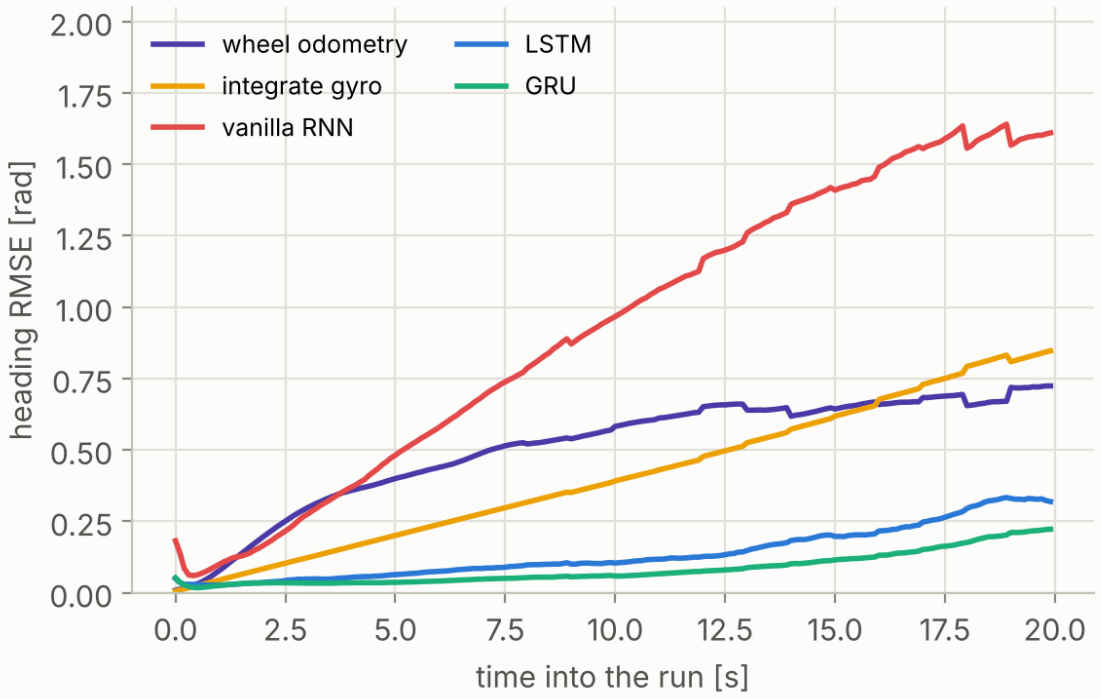
Pad
output at L−1: error 0.0
h_n: error 0.16 · BiLSTM: 0.10

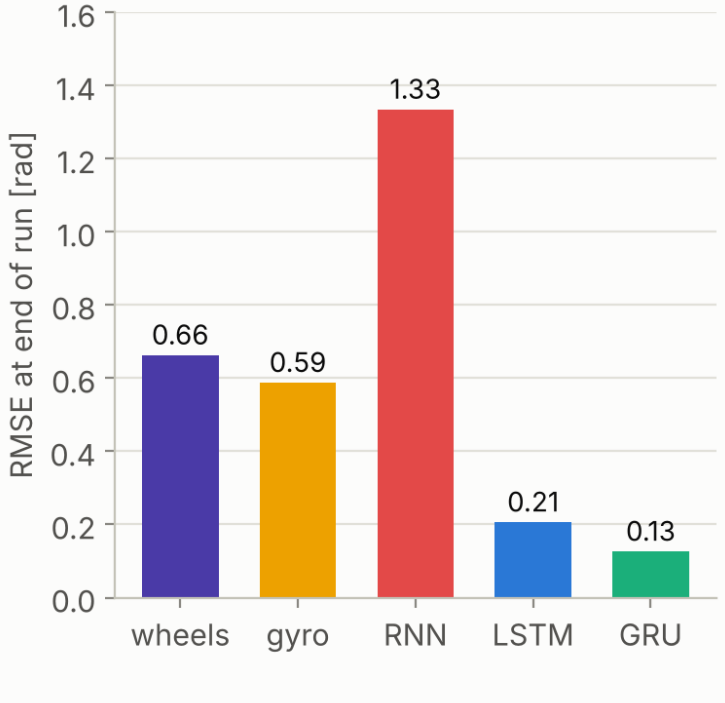
Pack
h_n: error 6.0×10^{-8} · BiLSTM: 8.9×10^{-8}
0.85 s per batch on CPU

Bucket by length
no padding exists, nothing to mask
0.10 s per batch → 9× faster

Fusing two sensors that lie in different ways

- The gyro drifts with its unknown bias; wheel odometry jumps at every slip. Each recurrent model sees all seven channels and predicts ψ at every step.
- The vanilla RNN ends 1.33 rad off, worse than either sensor alone. The GRU ends 0.13 rad off, 4.7× better than the best single sensor.

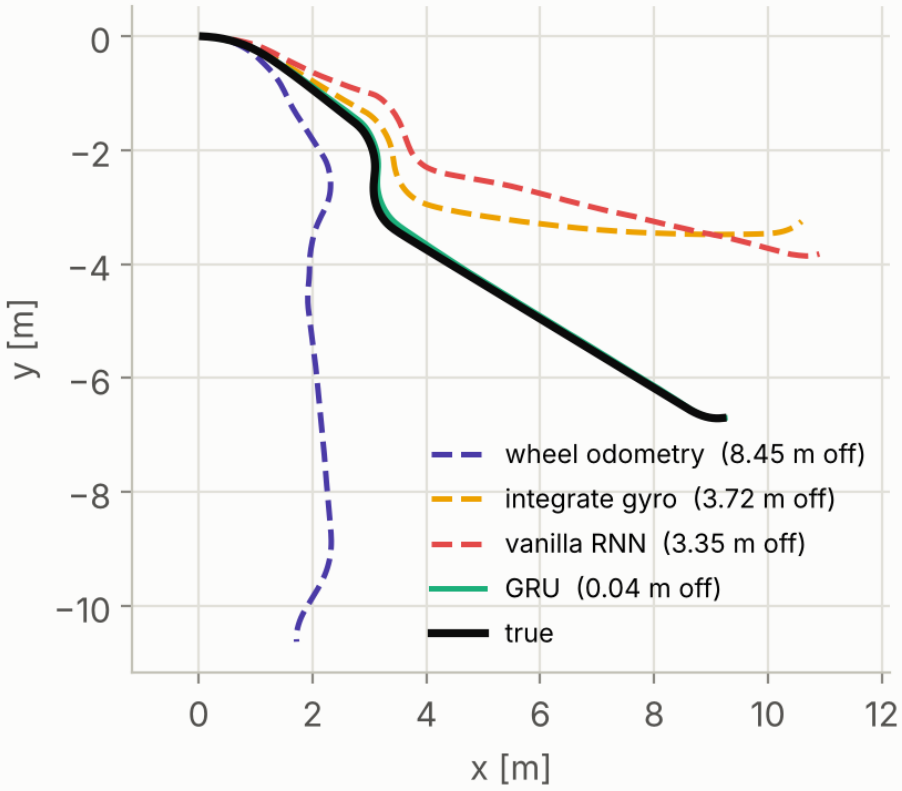


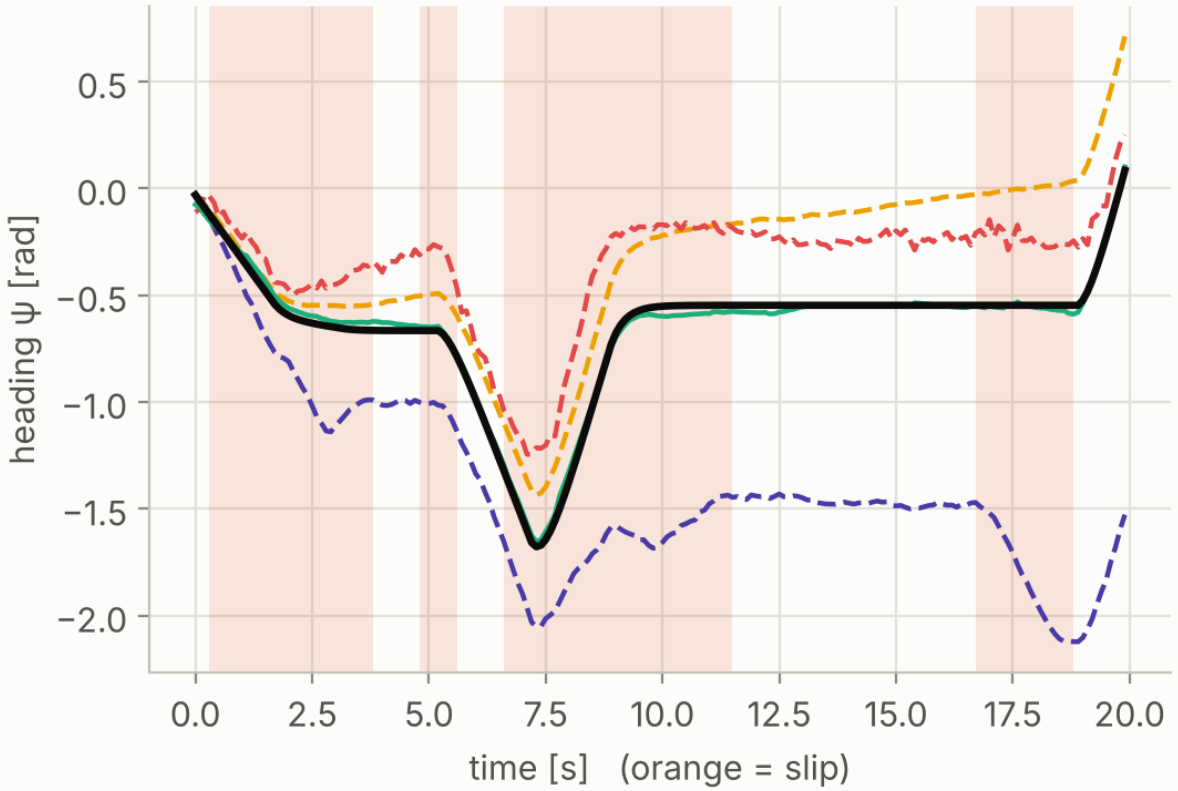


LSTM 0.21 rad, GRU 0.13 rad: both learned to trust the wheels when they agree with the gyro, and the gyro when they don't.

Where did the rover actually end up?

- Integrate the true speed along each heading estimate and the heading error becomes a position error, which is what a navigation stack actually cares about.



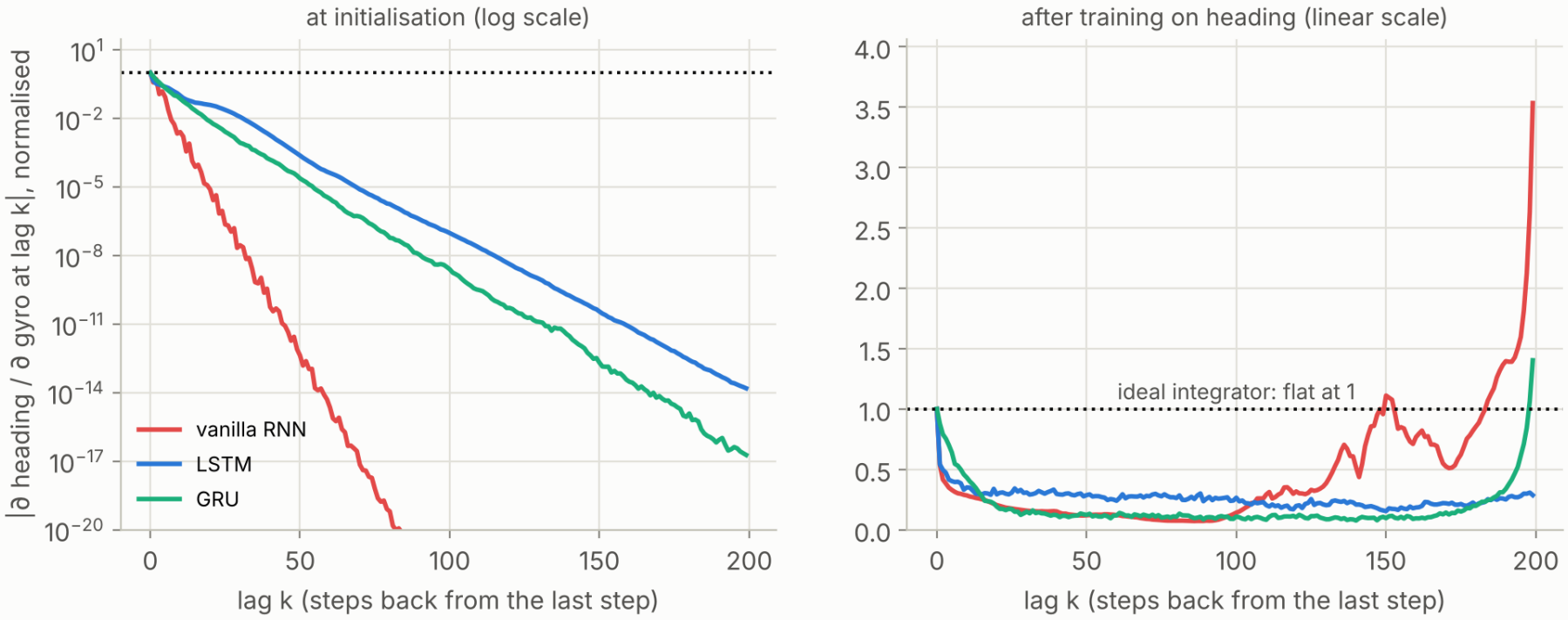


End-point error on this run: wheels 8.4 m · gyro 3.7 m · vanilla RNN 3.3 m · GRU 0.04 m.

BPTT

Why the vanilla RNN loses: measure the gradient

- For a perfect integrator ∂ψ̂_T/∂gyro_(T−k) is the same for every lag k: a flat line. Autograd lets us measure this curve for any trained model.
- At initialisation every model forgets: lag 100 is 1.2×10⁻24 for the RNN and 9.6×10⁻8 for the LSTM, even with its forget gate opened.

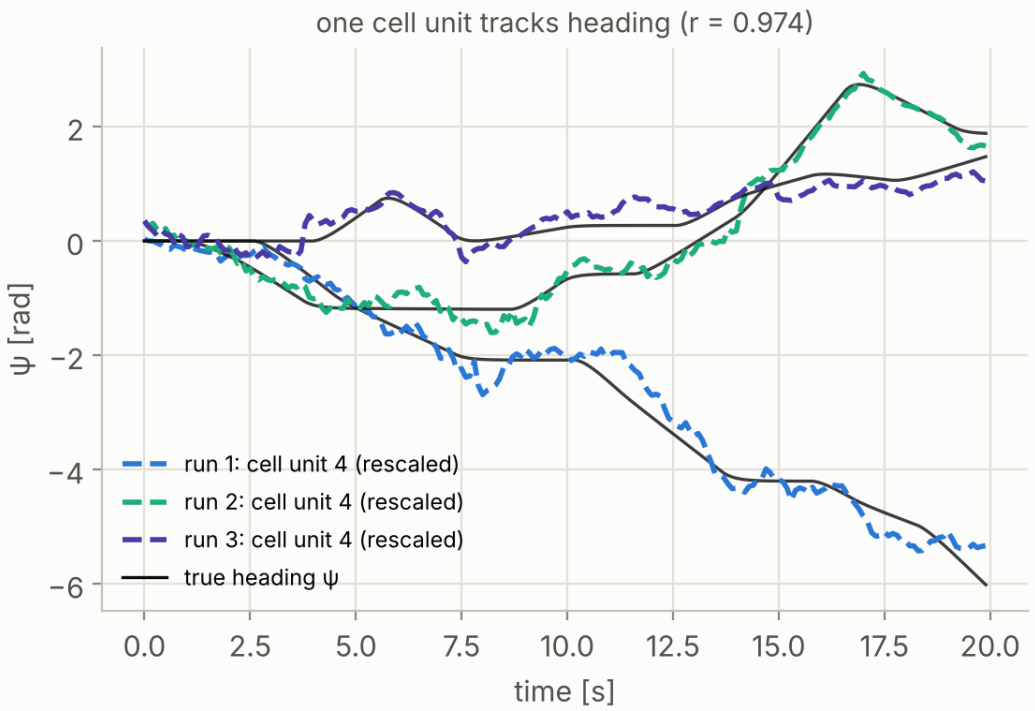


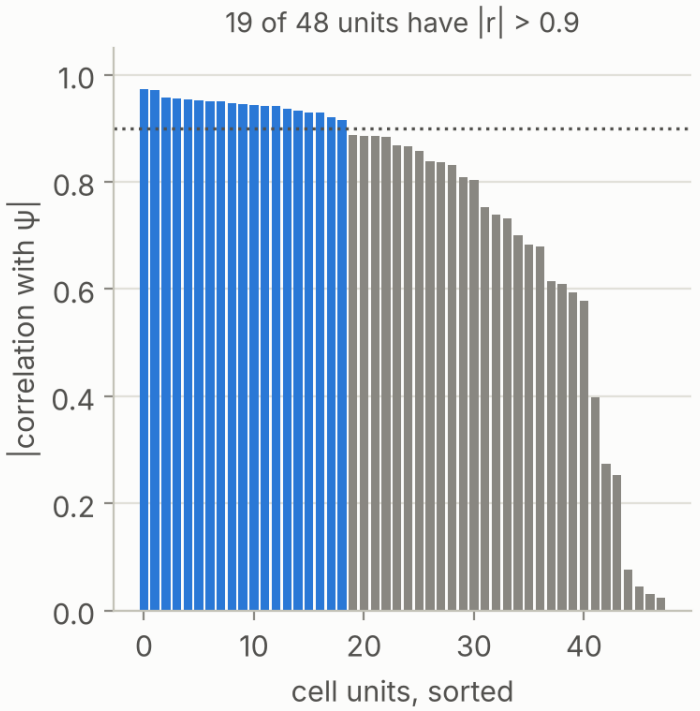
After training the LSTM holds 0.35 at lag 10 and 0.25 at lag 190: nearly flat. The RNN's curve is erratic. Gates make the long path learnable.

INTERPRETATION

Inside the trained LSTM: who carries the heading?

- Step the trained LSTM by hand with the four gate equations (it matches nn.LSTM to 6.0×10^{-7}), then correlate each cell unit c_t with the true heading ψ_t.
- 19 of 48 units have $|r| > 0.9$; the best reaches $r = 0.974$. Nobody labelled a heading unit; the loss created one.

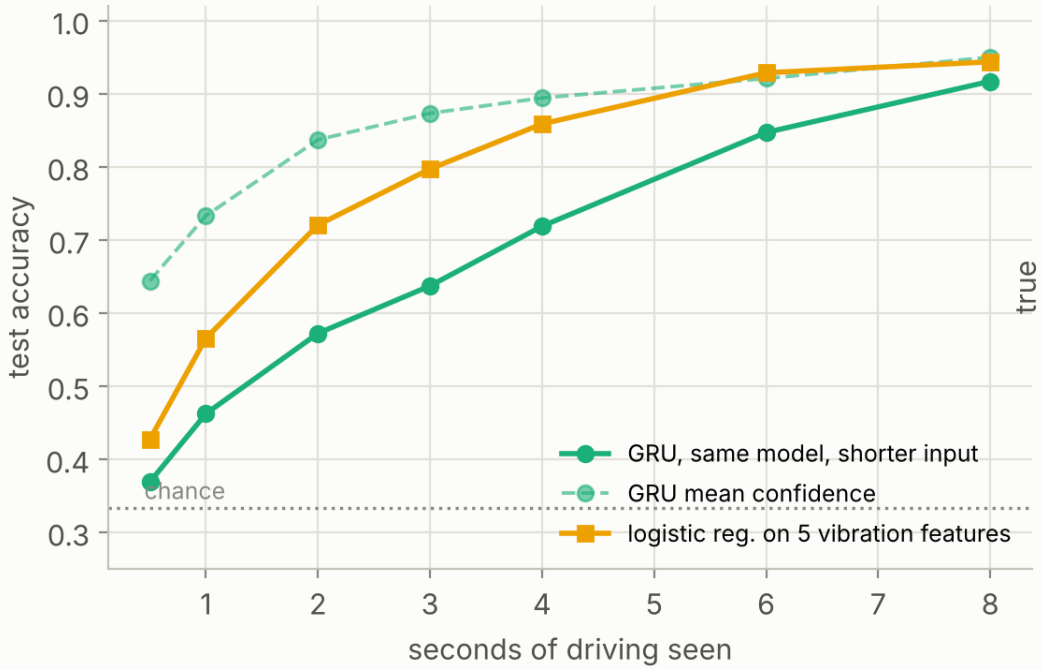


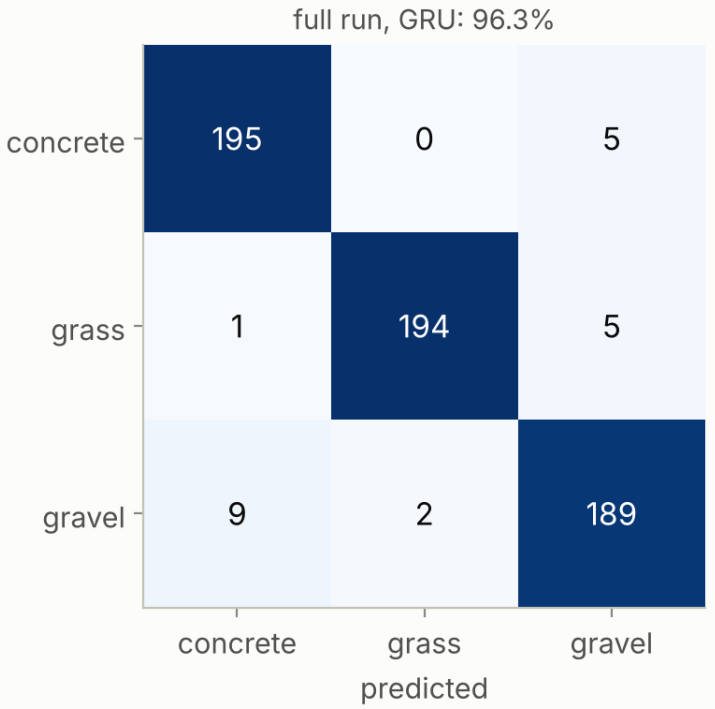


Its forget gate averages only 0.76, not ≈ 1: the network refreshes the stored heading every step instead of acting as a pure integrator.

Many-to-one: a belief that sharpens with evidence

- Concrete, grass and gravel differ only in the statistics of acc_z, so a single sample is ambiguous. Feeding the same causal model the first k steps shows its belief growing.
- The GRU reaches 96.3 % on full runs; five hand-made vibration features with logistic regression reach 97.2 %. The LSTM gets 81.8 %.

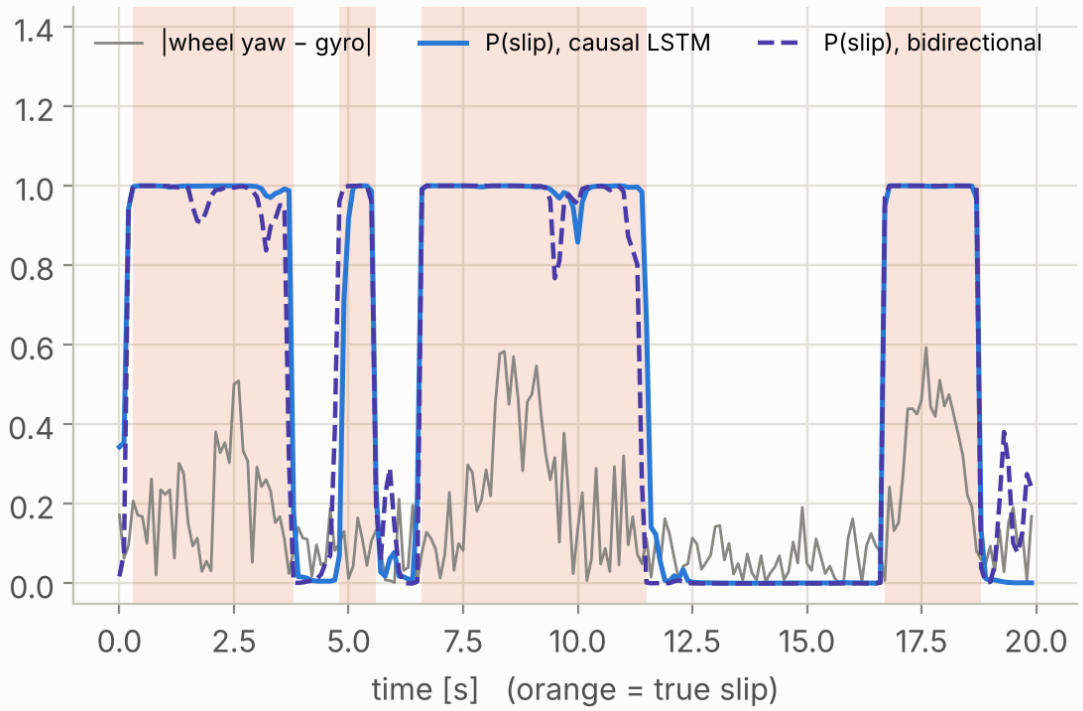




When physics already names the statistic, an RNN only has to match it. RNNs earn their keep where order and memory matter.

Many-to-many: wheel slip, online or offline?

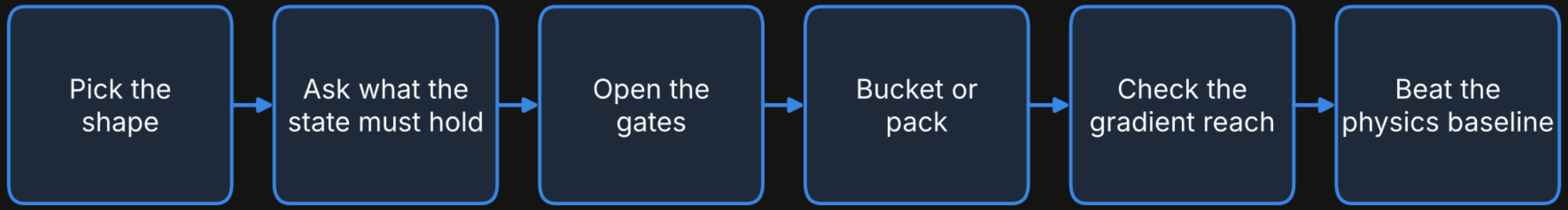
- A causal LSTM labels slip at every step using only the past; a bidirectional LSTM also reads the future, so it can only label logs after the run.
- Both beat a hand threshold rule by a wide margin (F1 0.972 vs 0.665). The bidirectional model scores 0.968 with twice the parameters.





The evidence for slip is visible as it happens, so the future adds nothing. The causal model detects onsets with a median latency of 0 steps.

What to take away



1. A hidden state is a state estimate. If the physics is $x_t = f(x_{t-1}, u_t)$, the recurrence is already there.
2. Pick the shape from where the answers are; pick the cell from how far back they must look.
3. Integration needs gain 1. Vanilla RNN 1.33 rad, GRU 0.13 rad on the same data.
4. Plot the gradient reach before tuning anything: it shows how far back a model can learn from.
5. A forward model's output at L−1 ignores padding; h_n and bidirectional layers do not. Pack, or bucket.
6. Bidirectional only when the future carries evidence and you can wait for it.
7. Always report the physics baseline. On terrain, five hand features tie the best RNN.

Generated by Claude and customized by

dew.ninja 2026

Notebook: mlpi26_Lect10_nbsup2.ipynb (8 exercises, references) · Next: Supplement B — where a fixed-size state runs out, and attention takes over